Computer Science 230
Computer Architecture and Assembly Language
Spring 2025

Assignment 1 (Part 2) (worth 6%)

Due: Thursday, Jan 30, 11:55 pm by Brightspace submission
(Late submissions are **not** accepted)

Please note that the Assignment 1 is in two parts: Part1 (worth 2%) is a set of multiple-choice questions distributed through Crowdmark; Part 2 (worth 6%) is this set of programming exercises, distributed through Brightspace.

Programming environment

For this assignment you must ensure your work executes correctly on the simulator within Microchip Studio 7 as installed on workstations in ECS 249. If you have installed AVR Studio on your own, then you are welcome to do much of the programming work on your machine. (The IDE is available at no charge from `https://bit.ly/3llENk7` but comes only in the Microsoft Windows flavor.) **You must allow enough time** to ensure your solutions work on the lab machines. If your submission fails to work on a lab machine, the fault is very rarely that the lab workstations. "It worked on my machine!" will be treated as the equivalent of "The dog ate my homework!"

Individual work

This assignment is to be completed by each individual student (i.e., no group work). Naturally you will want to discuss aspects of the problem with fellow students, and such discussion is encouraged. **However, sharing of code fragments is strictly forbidden without the express written permission of the course instructor.** If you are still unsure regarding what is permitted or have other questions about what constitutes appropriate collaboration, please contact me as soon as possible. (Code-similarity analysis tools will be used to examine submitted work.) The URLs of significant code fragments you have found and used in your solution must be cited in comments just before where such code has been used.

Objectives of this assignment

- Understand short problem descriptions for which an assembly-language solution is required.
- Use AVR assembly language to write solutions to small problems.
- Use AVR Studio to implement, simulate and test your solution. (We will not need to use the Arduino mega2560 boards for this assignment.)

- Hand draw a flowchart corresponding to your solution to the second problem.
- **You are not to use "AVR functions" in this assignment.**

Question 1: Addition of two sixteen-bit numbers

AVR has two add instructions ADD (add without carry) and ADC (add with carry) that can add two 8-bit numbers. These instructions take two registers as operands (they can be any of the 32 available), add their contents with or without carry, and store the result back in the first operand. Your objective is to extend this concept to add two 16 numbers. Obviously, to represent one 16-bit number, you would need two 8 bit registers. The AVR Architecture cannot directly add two 16-bit numbers stored in registers with a single instruction, but it can do it in two. You can start by adding the lower order bytes of our 16-bit numbers using the add instruction and use add with carry to add upper order bytes.

Your task for Question 1 is to complete the code in **`sixteen-bit-addition.asm`** that is provided to you. The code is to add two sixteen-bit numbers and move the resultant value to specified registers.

- The sixteen-bit operands for addition are in (r16, r17) and (r18, r19)
- The lower order bytes are r16 and r18. Upper order bytes are r17 and r19.
- The final sum should be moved to registers (r4, r5), r4 being the lower order byte.

Please read the ASM file **`sixteen-bit-addition.asm`** for more detail on what is required

Question 2: Resetting the right-most contiguous sequence of bits

Another common task when working with bit sequences is to identify and work with *contiguous set bits*. For example, consider the bit sequence shown below, with several of the set bits shown in a bold font:

`010`**`111`**`00`

We say that the bolded set bits constitute the *right-most contiguous set bits*. That is, bits 4, 3, and 2 are set. If we *reset the right-most contiguous set bits* of the example just given, the result is:

`010`**`000`**`00`

Your task for Question 2 is to complete the code in **`reset-rightmost.asm`** that is provided to you. Please read this file for more details on what is required.

- The bit sequence for which the right-most contiguous set bits must be reset is in `R16`.
- The result must be stored in `R1`.

Some test cases are provided to you in the provided assembly file.

You must also draw by hand the flowchart of your solution and submit a scan or smartphone photo of the diagram as part of your assignment submission; please ensure this is a JPEG or PDF named "`reset-rightmost-flowchart.jpg`" or "`reset-rightmost-flowchart.pdf`" depending on the file format you have chosen. (Please: No BMPs or Word files! We beg of you!)

Question 3: Two's Complement Conversion

As discussed in lecture Slide Set 2 (Numeration Systems), two's complement representation is used for negative numbers when using signed arithmetic in computers. On slides 38 and 39th, a method is described that uses a change sign rule by starting with a positive number, finding the rightmost bit that is set to 1 and then flipping all bits to the left of it. For example, consider the 8-bit binary equivalents of `+12` and `-12`:

```
          76543210 – Bit numbers (b7,b6, …, b0)
         ---------
+12      00001100
-12      11110100
```

To get the two's complement representation for -12, start with the binary equivalent of +12, find the right most bit that is set to 1 (in this case it is the bit in 3rd position from right, b2). Note that we number bits as (b7….b0) in a byte, b0 being the Least Significant Bit (LSB) and b7 being the Most Significant Bit (MSB). Keep all bits (including this bit) to the right of this bit intact and flip other bits.

```
+12 00001100 (right most bit that is set is b2)
-12 11110100 (flip all bits to the left of this bit)
```

Your task for Question 3 is to complete the code in `twos-complement.asm` that is provided to you. Please read this file for more details on what is required

- The values for which you will find the positive number to converted will be in register `r16`.
- The two's complement number will be stored in r17.
- Note that the original value provided in r16 must remain intact.

Some test cases are provided to you in the provided assembly file.

What you must submit

- Your completed work in the three source code files (`sixteen-bit-addition.asm`, `reset-rightmost.asm` and `twos-complement.asm`). Do

not change the names of these files! **Please do not submit any other AVR 7 Studio project files. Please do not submit ZIP files.**

- Your work must use the provided skeleton files. Any other kinds of solutions will not be accepted. (Again: Do not submit ZIP files containing the Microchip projects!)
- The scan / smartphone photo of your hand-drawn flowchart with the name of "`reset-rightmost-flowchart.jpg`" or "`reset-rightmost-flowchart.pdf`" depending on the format you have chosen.

Evaluation

- 3 marks: Sixteen-bit addition solution is correct for different pairs of input values (Q1).
- 3 marks: Reset Rightmost is correct for different byte values (Q2).
- 3 marks: Two's Complement solution is correct for a variety of byte values (Q3).
- 1 mark: All submitted code is properly formatted (i.e., indenting and comments are suitably used), and files correctly named.

Total marks: 10